

Трансляция PEG-грамматик в структурированное JSON-представление

Проект по курсу «Теория и реализация языков программирования»

Анна Новокшенова

Владимир Чайкин

Весенний семестр
2026

1 Постановка задачи

В данном проекте рассматривается задача трансляции описания грамматик класса PEG (Parsing Expression Grammars) в структурированное представление формата JSON.

Пусть G_{meta} — метаграмматика, описывающая синтаксис PEG-грамматик, а $L(G_{meta})$ — формальный язык, порождаемый этой метаграмматикой.

Требуется построить отображение f :

$$f : \Sigma^* \rightarrow \mathcal{J} \cup \{\perp\}$$

где:

- Σ — алфавит ASCII-символов входного файла;
- Σ^* — множество всех возможных входных цепочек символов;
- $\mathcal{J}$ — множество корректных JSON-документов заданной структуры;
- $\perp$ — состояние синтаксической ошибки, возвращаемое в случае, если входная строка $\omega \notin L(G_{meta})$.

Отображение f строится как композиция двух функций:

$$f = g \circ p$$

где:

- $p : \Sigma^* \rightarrow \mathcal{A} \cup \{\perp\}$ — функция синтаксического анализа (парсер), преобразующая входную строку в абстрактное синтаксическое дерево (AST) либо возвращающая ошибку, где $\mathcal{A}$ — множество корректных AST, представляющих структуру PEG-грамматики;
- $g : \mathcal{A} \rightarrow \mathcal{J}$ — функция сериализации, преобразующая AST в JSON-документ заданного формата.

1.1 Входные данные

На вход алгоритм должен принимать цепочку символов $\omega \in \Sigma^*$, представляющая собой текстовое описание PEG-грамматики. При этом корректная входная строка должна удовлетворять метаграмматике G_{meta} , для всех же $s \notin L(G_{meta})$ результатом работы алгоритма должен являться $\perp$.

Ниже приведено формальное определение G_{meta} в расширенной нотации Бэкуса–Наура.

Листинг 1: Метаграмматика G_{meta} (синтаксис входного PEG-файла)

```
Grammar      ::= Rule+
Rule         ::= Identifier "=" Expression
Expression   ::= Sequence ("/" Sequence)*
Sequence     ::= Prefix+
Prefix       ::= ("&" / "!" )? Suffix
Suffix       ::= Primary ("*" / "+" / "?" )?
Primary      ::= Identifier
               | Literal
               | CharClass
               | "."
               | "(" Expression ")"
Literal      ::= '"' [^"]* '"'
CharClass    ::= "[" [^\]]* "]"
Identifier   ::= [a-zA-Z_] [a-zA-Z0-9_]*
```

1.2 Выходные данные

На выходе алгоритма ожидается строка $s \in \mathcal{J}$, представляющая собой JSON-документ фиксированной структуры.

Документ должен представлять из себя JSON-массив объектов, каждый из которых соответствует одному правилу заданной PEG-грамматики.

Объект правила содержит два обязательных поля:

- "NT" — строка, имя нетерминала (левая часть правила);
- "expr" — объект, описывающий PEG-выражение в правой части правила. Каждый объект выражения содержит обязательное строковое поле "type", определяющее тип выражения. Остальные поля зависят от типа (подробнее описано в Таблице 1).

PEG-конструкция	Поле "type"	Дополнительные поля
Литерал "..."	"literal"	"value": "строка"
Идентификатор Name	"nonterminal"	"name": "имя"
Класс символов [...]	"class"	"value": "содержимое"
Точка .	"any"	—
Последовательность A B	"sequence"	"elements": [массив выражений]
Выбор A / B	"choice"	"options": [массив выражений]
Повторение A*	"zero_or_more"	"expr": {выражение}
Повторение A+	"one_or_more"	"expr": {выражение}
Опциональность A?	"optional"	"expr": {выражение}
Предикат &A	"and"	"expr": {выражение}
Предикат !A	"not"	"expr": {выражение}

Таблица 1: Типы выражений и их JSON-представление

2 Метод

Предлагаемый для решения поставленной задачи алгоритм, как уже было отмечено, разделен на два ключевых этапа: синтаксический анализ (парсинг) входного описания грамматики и последующая сериализация полученного AST в текстовый формат JSON.

2.1 Парсинг

Данная часть алгоритма реализована в модуле `PEGParser.hs`.

В основе разбора входной строки лежит метод синтаксического анализа, известный как рекурсивный спуск с возвратом, реализованный с помощью библиотеки парсер-комбинаторов `Parsec`. Данный метод естественным образом выражается через парсерные комбинаторы — функции высшего порядка, строящие сложные парсеры из простых базовых.

Комбинатор	Описание
<code>char c</code>	Парсит ровно один символ <code>c</code>
<code>oneOf s</code>	Парсит любой один символ из строки <code>s</code>
<code>many p</code>	Применяет парсер <code>p</code> ноль или более раз
<code>many1 p</code>	Применяет парсер <code>p</code> один или более раз
<code>try p</code>	Пробует применить <code>p</code> ; в случае неудачи откатывает состояние ввода к исходному
<code>p < > q</code>	Оператор выбора: пробует <code>p</code> , при неудаче — <code>q</code>
<code>notFollowedBy p</code>	Проверяет, что <code>p</code> не применим, не потребляя ввода

Таблица 2: Базовые комбинаторы библиотеки `Parsec`

2.1.1 Обработка пробельных символов

Для корректной обработки пробельных символов алгоритм разделяет их на два класса:

1. Горизонтальные пробелы (пробел, табуляция, возврат каретки) — игнорируются внутри выражений. За их обработку отвечает функция `ws`, определённая как `skipMany (oneOf " \t\r")`;

2. Вертикальные пробелы (перевод строки) — служат разделителями правил. Функция `endOfRule` требует наличия хотя бы одного перевода строки между соседними правилами.

2.1.2 Парсинг PEG-выражений

Парсер строится как взаимно-рекурсивная система функций, каждая из которых соответствует одному уровню приоритета операторов PEG. Ниже приведен их список в порядке от высшего приоритета к низшему (то есть от наиболее глубокого уровня рекурсии к верхнему):

1. `primary` — атомарные выражения: строковые литералы в двойных кавычках, идентификаторы, классы символов `[...]`, точка `.`, а также выражения в круглых скобках;
2. `suffix` — применяет к результату `primary` операторы повторения `*`, `+`, `?` (если они есть);
3. `prefix` — применяет к результату `suffix` предикаты `&` и `!` (если они есть);
4. `sequenceExpr` — собирает последовательность из одного или более `prefix`-элементов, следующих друг за другом в пределах одной строки. Использует комбинатор `notFollowedBy (char '\n')` для предотвращения перехода на следующую строку;
5. `expression` — собирает упорядоченный выбор из нескольких `sequenceExpr`, разделённых оператором `/`.

2.1.3 Парсинг грамматики

Грамматика в целом парсится функцией `grammar`:

1. Вызывается `ws` для пропуска пробелов перед первым правилом;
2. Парсится первое правило функцией `rule`, которая считывает идентификатор, знак `=` и выражение;
3. Вызывается `many (endOfRule » rule)` — парсинг остальных правил, каждое из которых отделено от предыдущего хотя бы одним переводом строки;
4. Вызывается `eof` для проверки, что весь вход обработан.

2.2 Сериализация

Данная часть алгоритма реализована в модуле `JSONEporter.hs`.

Функция сериализации `grammarToJSON` выполняет рекурсивный обход построенного AST и генерирует JSON-строку в соответствии с заданным форматом (раздел 1.2).

2.2.1 Обход дерева выражений

Центральная функция `exprToJSON` реализует рекурсивный обход типа `Expr`. Для каждого из конструкторов она формирует JSON-объект как строку, следуя единому шаблону:

1. Открывается фигурная скобка `{`;
2. Записывается обязательное поле `"type"` со строковым значением, уникальным для каждого конструктора;

3. В зависимости от конструктора записываются дополнительные поля (см. Таблица 1);
4. Если дополнительное поле содержит вложенные выражения, для каждого из них рекурсивно вызывается `exprToJSON`;
5. Закрывается фигурная скобка `}`.

2.2.2 Сериализация правил и грамматики

Функция `ruleToJSON` формирует JSON-объект для одного правила: записывает поле `"NT"` с именем нетерминала и поле `"expr"`, значение которого получает вызовом `exprToJSON`. Функция `grammarToJSON` оборачивает результаты `ruleToJSON` для всех правил грамматики в JSON-массив: открывающая квадратная скобка, разделённые запятыми объекты правил, закрывающая квадратная скобка.

2.2.3 Экранирование строковых значений

Строковые значения (имена нетерминалов, содержимое литералов и классов символов) перед вставкой в JSON проходят обработку функцией `escapeJSON`. Она посимвольно обходит строку и заменяет специальные символы на их escape-последовательности: двойная кавычка `"` заменяется на `\`, обратный слеш `\` — на `\\`, перевод строки — на `\n`, табуляция — на `\t`. Все остальные символы сохраняются без изменений.

2.2.4 Форматирование списков

При сериализации конструкторов `Sequence` и `Choice`, а также списка правил грамматики, элементы необходимо разделять запятыми. Наивное решение — добавлять запятую после каждого элемента — приводит к появлению висячей запятой (*trailing comma*) после последнего элемента, что делает JSON невалидным.

Для решения этой проблемы используется вспомогательная функция `joinWithComma`, реализующая рекурсивное разделение:

1. Для пустого списка возвращается пустая строка.
2. Для списка из одного элемента возвращается строковое представление этого элемента без запятой.
3. Для списка из двух и более элементов первый элемент отделяется запятой от результата рекурсивного вызова на хвосте списка.

Данная функция применяется при формировании массива правил грамматики, массива `"elements"` последовательности и массива `"options"` выбора.

2.3 Итоговый алгоритм

Общий алгоритм обработки входного файла, реализованный в модуле `Main.hs`:

1. Прочитать содержимое входного файла в строку.
2. Применить функцию парсинга `parsePEG` к полученной строке.
3. Если парсинг завершился ошибкой — вывести сообщение об ошибке в стандартный поток ошибок (`stderr`) и завершить программу с ненулевым кодом возврата.

4. Если парсинг успешен — применить функцию сериализации `grammarToJSON` к полученному абстрактному синтаксическому дереву.
5. Вывести результирующую JSON-строку в стандартный поток вывода (`stdout`).

Данный алгоритм реализует отображение $f : \Sigma^* \rightarrow \mathcal{J} \cup \{\perp\}$, определённое в постановке задачи (раздел 1), как композицию двух функций: $f = g \circ p$, где p — функция парсинга, g — функция сериализации.

3 Реализация

Исходный код проекта представлен на GitHub¹. Там же можно найти тесты, проверяющие корректность метода и правильность его реализации.

¹Code at https://github.com/DecembersMiracle/peg2json_TRPL_project_2026